Project Report

Theme: Fine-Tuning Gemma 4 for Children's Story Generation

This NLP project focuses on domain adaptation by fine-tuning Google's Gemma 4 (E2B-IT) model using QLoRA to generate educational and engaging children's stories. The pipeline includes model quantization, merging, and deployment using GGUF for efficient local inference.

Group Members:

Benhammadi Lokmane

Khedim Youcef

May 4, 2026

Contents

List of Figures

1. Introduction and Problem Statement

1.1. The Challenge: Domain-Specific Text Generation

Large Language Models (LLMs) are incredibly capable zero-shot generalists. However, when tasked with creating highly specific content—such as educational stories tailored to exact age groups—they often drift into generic language, hallucinate complex vocabulary, or fail to adhere to structural constraints.

Two primary challenges define this project:

- **Style Adaptation:** Adjusting the model's tone to be engaging, safe, and comprehensible for young children, categorized strictly by age levels.
- **Resource Constraints:** Fine-tuning billions of parameters requires substantial compute memory. Developing a pipeline capable of running on accessible hardware (like Kaggle's dual T4 GPUs) is critical.

1.2. Our Approach: QLoRA Fine-Tuning and Edge Deployment

We fine-tune Google's `gemma-4-E2B-it` model on a curated children's story dataset. To bypass hardware limitations, we utilize 4-bit Quantized Low-Rank Adaptation (QLoRA). Once trained, the adapters are merged back into the base model and converted into a highly optimized `.gguf` format, enabling fast, local inference on edge devices via `llama.cpp`.

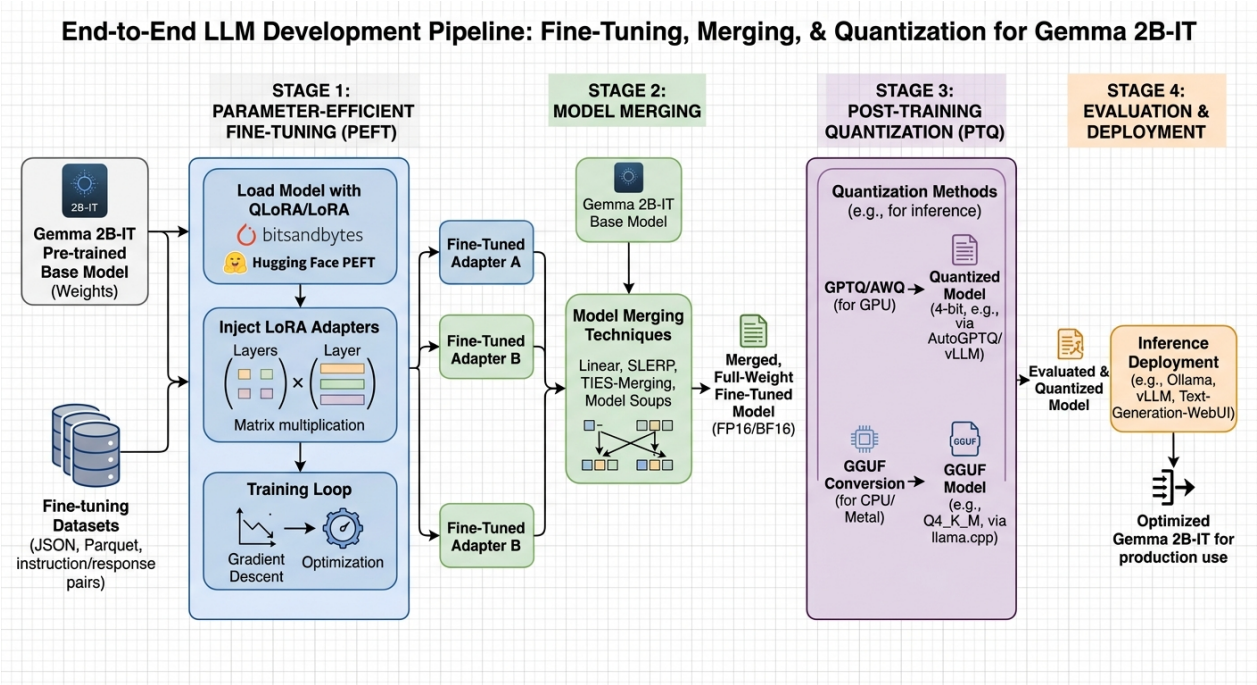


Figure 1: End-to-end LLM fine-tuning, merging, and quantization pipeline.

2. Background: Parameter-Efficient Fine-Tuning

2.1. The Memory Bottleneck

Training an LLM requires memory for model weights, gradients, optimizer states, and activations. A 2-billion parameter model in 16-bit precision requires roughly 4GB just to hold the weights, but over 16GB to train fully. This makes full fine-tuning inaccessible for consumer GPUs.

2.2. Quantization and QLoRA

To solve this, we employ **QLoRA**. The base model is loaded in 4-bit precision (NF4), drastically reducing the memory footprint. Instead of updating the original weights, QLoRA injects small, trainable "adapter" matrices into the model's layers. During training, only these adapter weights are updated while the base model remains frozen.

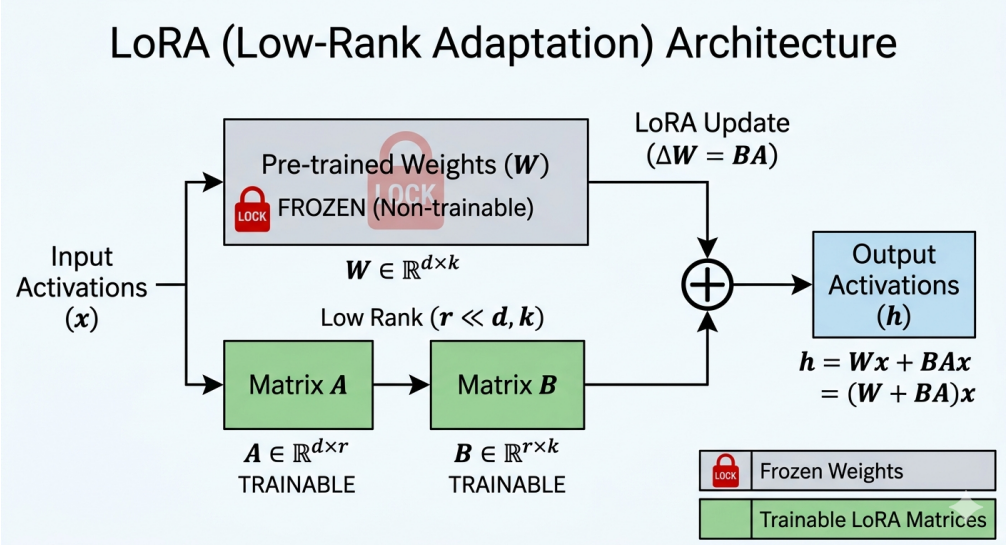


Figure 2: LoRA architecture: small rank decomposition matrices are trained while the pre-trained weights remain frozen.

3. Model Architecture: Gemma 4

3.1. Overview of Gemma 4 E2B-IT

Gemma is a family of lightweight, state-of-the-art open models built from the same research and technology used to create Google's Gemini models. We selected the `gemma-4-E2B-it` (Instruction Tuned) variant, which features an effective parameter count of 2.3 billion.

3.2. Why Gemma 4 E2B?

Three reasons drove this selection:

- **High Capacity-to-Size Ratio:** At 2.3B effective parameters, it offers deep reasoning capabilities while remaining small enough to fit comfortably on a single 16GB GPU during training.
- **Instruction Tuning:** The base model is already aligned to follow instructions and multi-turn conversations, meaning it requires less training data to learn the formatting of a prompt-response task.
- **Massive Vocabulary:** The model possesses a vocabulary size of 256,000 tokens, giving it exceptional multilingual and structural generation capacity.

4. Dataset

4.1. Source and Formatting

We utilized the `ajibawa-2023/Children-Stories-Collection` dataset from HuggingFace. The dataset consists of story prompts and corresponding stories suitable for children.

To teach the model to adapt its vocabulary, we prefixed each prompt with an age-level constraint. The training samples were formatted into Gemma's native instruction format:

```
<start_of_turn>user
Level: Age5-6 - Short sentences.
Write a story about a brave little toaster.<end_of_turn>
<start_of_turn>model
Once upon a time, there was a little silver toaster. He lived in a warm kitchen...<
    end_of_turn>
```

4.2. Dataset Statistics

We subsampled 10,000 stories for this fine-tuning pass, splitting 95% for training and 5% for evaluation.

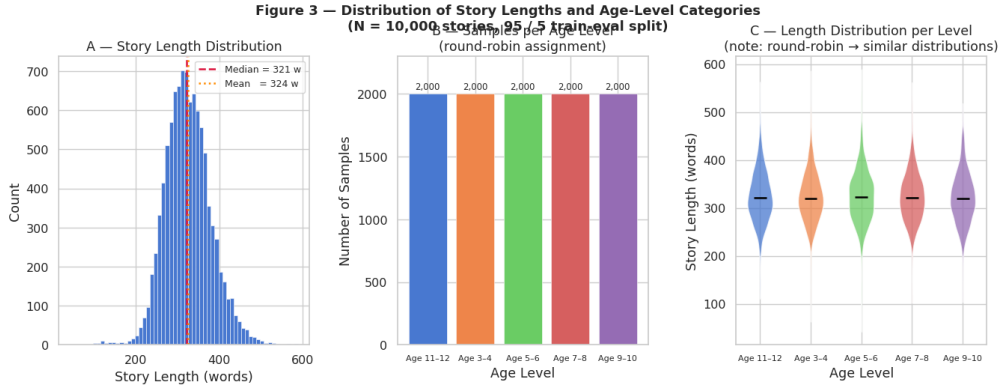


Figure 3: Distribution of story lengths and age-level categories in the training sample.

5. Training Methodology

5.1. Configuration and Hardware

Training was orchestrated using the HuggingFace `Trainer` API on a Kaggle environment equipped with dual NVIDIA T4 GPUs.

Key training parameters:

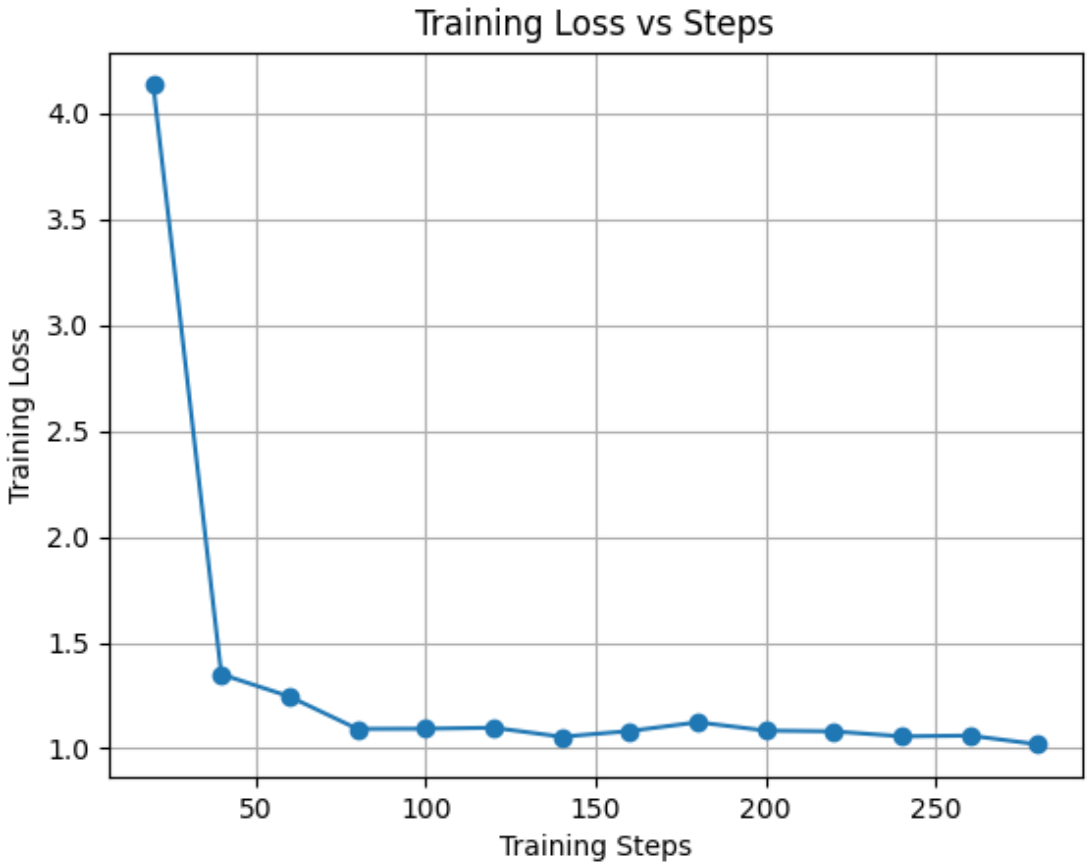
Training Arguments

```
training_args = TrainingArguments(
    per_device_train_batch_size=1,
    gradient_accumulation_steps=16,
    optim='paged_adamw_8bit',
    learning_rate=2e-4,
    fp16=True,
    gradient_checkpointing=True,
    eval_strategy='no' # Disabled to prevent OOM
)
```

5.2. Overcoming Memory Constraints

Due to Gemma 4's massive 256K token vocabulary, calculating cross-entropy loss over large sequences creates a massive memory spike. We mitigated this by:

1. Enforcing `batch_size = 1` with high gradient accumulation (16 steps) to simulate a batch size of 16.
2. Enabling `gradient_checkpointing` to trade computation time for VRAM savings.
3. Setting `PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True` to prevent CUDA memory fragmentation.



we can note that the loss was a bit fluctuating over the steps but it still was decreasing overall, which is a good sign that the model was learning to adapt to the new task. The fluctuations could be due to the small batch size and the complexity of the task, but the downward trend indicates that the fine-tuning process was effective in improving the model's performance on generating children's stories.

Figure 4: Training loss curve over the 1-epoch fine-tuning run.

6. Model Export and Deployment

6.1. Merging the Adapters

QLoRA results in standalone adapter weights. For efficient deployment, we reloaded the base model in FP16 precision on the CPU, injected the LoRA weights, and invoked `merge_and_unload()` to fuse them permanently.

6.2. GGUF Quantization via llama.cpp

To deploy the model locally, we converted the merged model into the GGUF (GPT-Generated Unified Format) standard using `llama.cpp`. The FP16 model was further quantized down to `Q4_K_M` (4-bit, medium quantization). This reduced the model's disk footprint to approximately 1.5 GB, allowing it to run entirely on consumer CPU RAM with minimal loss in generative quality. Another problem we faced was the management of the kaggles limited ram and storage resources so we had to implement a pipeline that constantly monitored and cleaned up the unnecessary files at/after each stage of the conversion and quantization process to ensure that we didn't run out of space during the process.

```
============================================================
STAGE 1: TRAIN
============================================================
✅ Final adapter found on HF Hub — skipping training.
   Downloading adapter to /kaggle/working/lora_adapter...
Loading widget...
Loading widget...
   Adapter ready locally.

✅ STAGE 1 COMPLETE
```

```
============================================================
STAGE 2: MERGE TO DISK
============================================================

  [1/4] Clearing HF cache to make room on disk...
  💾 Disk free [after cache wipe]: 20.8 GB

  [2/4] Loading base model (FP16)...
Loading widget...
 [transformers] `torch_dtype` is deprecated! Use `dtype` instead!
Loading widget...
Loading widget...
Loading widget...
  Unwrapped 232 Gemma4ClippableLinear modules
  [3/4] Merging LoRA weights...
  [4/4] Saving merged model to /kaggle/working/gemma_merged...
Loading widget...

✅ STAGE 2 COMPLETE
   Disk used: 10.4 GB
   RAM free: 15.0 GB
```

Figure 5: The llama.cpp conversion and quantization process log (part 1).

```
============================================================
STAGE 3: CONVERT & QUANTIZE
============================================================
  💾 Disk free [before GGUF conversion]: 10.5 GB

  Building llama.cpp (one-time, ~5 min)...
Cloning into '/kaggle/working/llama.cpp'...
-- The C compiler identification is GNU 11.4.0
-- The CXX compiler identification is GNU 11.4.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/cc - skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: /usr/bin/c++ - skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Found Git: /usr/bin/git (found version "2.34.1")
-- The ASM compiler identification is GNU
-- Found assembler: /usr/bin/cc
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD
CMAKE_BUILD_TYPE=Release
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD - Success
-- Found Threads: TRUE
-- Warning: ccache not found - consider installing it for faster compil
-- CMAKE_SYSTEM_PROCESSOR: x86_64
-- GGML_SYSTEM_ARCH: x86
-- Including CPU backend
```

```
[ 97%] Building CXX object tests/CMakeFiles/test-chat.dir/get-model.cpp.o
[ 98%] Building CXX object tests/CMakeFiles/test-gguf-model-data.dir/test-gguf-model-data.cp
[ 98%] Linking CXX executable ../bin/test-gguf-model-data
[ 98%] Built target test-gguf-model-data
[ 98%] Building CXX object tests/CMakeFiles/test-quant-type-selection.dir/test-quant-type-se
[ 98%] Building CXX object tests/CMakeFiles/test-quant-type-selection.dir/get-model.cpp.o
[ 99%] Linking CXX executable ../bin/test-quant-type-selection
[ 99%] Built target test-quant-type-selection
[ 99%] Building CXX object tests/CMakeFiles/export-graph-ops.dir/export-graph-ops.cpp.o
[ 99%] Linking CXX executable ../bin/export-graph-ops
[ 99%] Built target export-graph-ops
[100%] Linking CXX executable ../../bin/llama-server
[100%] Built target llama-server
[100%] Linking CXX executable ../bin/test-chat
[100%] Built target test-chat
  ✅ llama.cpp built.

  Converting /kaggle/working/gemma_merged → FP16 GGUF...
```

Figure 6: The llama.cpp conversion and quantization process log (part 2).

```
  ✅ FP16 GGUF: 9.27 GB
  Freeing /dev/shm (~9 GB RAM)...
  💾 Disk free [after /dev/shm free]: 11.2 GB

  Quantizing to Q4 K M...
```

```
main: quantize time = 193147.70 ms
main:    total time = 193147.70 ms
  ✅ Q4_K_M GGUF: 3.42 GB
  💾 Disk free [end of Stage 3]: 17.0 GB

✅ STAGE 3 COMPLETE
```

```
 ============================================================
 STAGE 4: UPLOAD
 ============================================================
   Uploading 3.42 GB to khedim/NLP-MINI-PROJECT...
Loading widget...
Loading widget...
   ✅ Upload complete.
   💾 Disk free [final]: 20.4 GB

 ============================================================
 🎉 ALL DONE!
    GGUF : https://huggingface.co/khedim/NLP-MINI-PROJECT
    LoRA : https://huggingface.co/khedim/NLP-MINI-PROJECT-adapter
 ============================================================
```

Figure 7: The llama.cpp conversion and quantization process log (part 3).

7. Evaluation and Results

7.1. Qualitative Examples

Post fine-tuning, the model reliably adhered to the age-level prefixes, adjusting its sentence complexity and vocabulary seamlessly.

Figure 8: Comparison of generated stories for Age 3-4 vs Age 11-12 from the same prompt.

8. Discussion and Limitations

While the fine-tuned model successfully generated formatted, age-appropriate children's stories, several challenges and limitations remain:

- **Vocabulary Size Overhead:** The 256k vocabulary size of Gemma 4 makes cross-entropy loss computation extremely expensive in VRAM, restricting training batch sizes and evaluation strategies.
- **Catastrophic Forgetting:** By fine-tuning purely on children's stories, the model's general instructional capability may degrade outside of this specific domain.
- **Hallucinations:** Without grounding constraints, the model may occasionally generate nonsensical plot resolutions. Future work could involve RAG (Retrieval-Augmented Generation) to ground the stories in specific educational facts.

9. Technologies Used

9.1. PyTorch & Transformers

PyTorch powers the underlying tensor math, while the HuggingFace Transformers library handles model loading, tokenization, and the training loop abstraction.

9.2. PEFT & BitsAndBytes

The PEFT library manages the LoRA adapter injection, while BitsAndBytes enables the NF4 precision loading essential for QLoRA on consumer hardware.

9.3. llama.cpp

Written in pure C/C++, llama.cpp provides the engine for converting the PyTorch model into the highly optimized GGUF format for edge inference.

9.4. Kaggle

Kaggle's cloud environment, specifically the dual T4 GPU accelerators, served as the primary hardware infrastructure for the fine-tuning process.